

Imam Mohammad Ibn Saud Islamic University

Applied College

Network Technologies Diploma

***Deployment and Modernization of Legacy PS2 Online Server
Infrastructure Using Docker***

Student Names:

[Ahmad Meshal Bin Obaid] — [446014143]

[Khattab Abdullah Asiri] — [446018941]

[Nawaf Mohammed Almutairi] — [446003153]

[Abdulrahman Mohammed Alsuaideen] — [446000755]

[Nawaf Abdullelah Bin Saaed] — [446004561]

Supervisor: Dr. Saad Ali Saad Al-Ahmari

Year: 2026

Acknowledgment

The members of this group declare that this report is a work done by the members of the group except the information that has been legitimately obtained from literature reviews, company, or university sources. According to the university policy of plagiarism, all the sources have been cited.

Academic Number	Tasks	Percentage %	Signature

Tools and AI Assistance Disclosure: Parts of this project — including Docker configuration scaffolding, English-language editing, and diagram drafting — were produced with the assistance of an AI writing/coding assistant under direct student supervision. All technical decisions, testing, debugging, and validation were performed and verified by the student authors. This disclosure is provided in the interest of academic transparency; please refer to institutional policy for the applicable citation format.

Recognition

We would like to thank Imam Mohammad Ibn Saud Islamic University and the Applied College, and our supervisor Dr. Saad Ali Saad Al-Ahmari for his guidance and feedback throughout this project.

We also acknowledge Corbin [8], whose publicly shared documentation of the Biohazard Outbreak private server setup was our first introduction to this subject; the developer known as "ghostline" (GitLab handle `gh0stl1ne`), whose open-source Bioserver1, Bioserver2, and DNASrep projects — licensed under the GNU AGPL-3.0 — provided the server codebase deployed in this work; and the `obsrv.org` project, whose protocol reconstruction and client-side configuration guidance were essential references.

Abstract

This project focuses on the deployment and modernization of a private online server infrastructure for the classic PlayStation 2 games Biohazard Outbreak File #1 and File #2. Capcom retired the dedicated game servers in stages — File #2 in March 2007, the North American File #1 servers in December 2007, and the Japanese File #1 service in June 2011 — while Sony's DNAS authentication layer continued operating until its own shutdown in April 2016. This project builds upon an existing open-source server emulation codebase publicly released by gh0stl1ne (licensed under the GNU AGPL-3.0), and our work centered around successfully deploying, troubleshooting, and modernizing this infrastructure using Docker containerization. The original codebase targeted outdated environments and required significant compatibility work to run on modern systems, including migrating deprecated PHP MySQL functions to MySQLi, resolving MariaDB authentication and legacy JDBC driver conflicts, and rebuilding Apache HTTP Server from source with a custom OpenSSL 1.0.2 build to support the legacy SSL ciphers required by the PS2. Wireshark was used throughout to analyze PS2 network traffic and diagnose connectivity issues between the game client and the deployed servers. Finally, secure remote connectivity between players was established using a WireGuard-based VPN (Tailscale), successfully enabling verified online multiplayer between geographically separated players.

Keywords: containerization, Docker, legacy systems, server emulation, PlayStation 2, network protocol analysis, VPN

Table of Contents

List of Abbreviations	6
Chapter 1: Introduction	7
Chapter 2: Literature Review	12
Chapter 3: System Analysis and Design	18
Chapter 4: Application	27
Chapter 5: Test and Results	40
Chapter 6: Conclusion and Future Work	48
References	52
List of Figures	53
List of Tables	55
Appendix	55

List of Abbreviations

Abbreviation	Meaning
DNAS	Dynamic Network Authentication System (Sony)
KDDI	Matchmaking service operator used by Sony for PS2 online titles
JDBC	Java Database Connectivity
SSL/TLS	Secure Sockets Layer / Transport Layer Security
VPN	Virtual Private Network
JAR	Java Archive
MOTD	Message of the Day
DHE / EDH	Diffie-Hellman Ephemeral (key exchange; two historical OpenSSL naming conventions for the same mechanism)
APR	Apache Portable Runtime
FCGI	FastCGI
AGPL	GNU Affero General Public License
PS2	PlayStation 2
NFR / FR	Non-functional Requirement / Functional Requirement

Chapter 1: Introduction

1. Introduction

1.1 Introduction

The PlayStation 2, released by Sony in 2000, was one of the most successful gaming consoles in history, selling over 155 million units worldwide. Among its most celebrated titles was Biohazard Outbreak (known as Resident Evil Outbreak in Western markets), a survival horror game developed by Capcom that introduced a groundbreaking online cooperative multiplayer experience for console gaming at the time.

The online infrastructure supporting these games was built upon three interconnected server layers: Sony's Dynamic Network Authentication System (DNAS), which verified the authenticity of both the console hardware and game software; a KDDI-operated matchmaking service, which managed player accounts and connected players to game lobbies; and Capcom's dedicated game servers, which handled real-time multiplayer sessions. Together, these systems allowed players worldwide to cooperate online in fighting zombie outbreaks across two titles: Biohazard Outbreak File #1 (2003) and Biohazard Outbreak File #2 (2004).

The two layers of this infrastructure were retired at different times. Capcom retired the dedicated Biohazard Outbreak game servers in stages rather than all at once: the File #2 servers closed on March 31, 2007; the North American File #1 servers followed on December 31, 2007; and the Japanese File #1 service — the regional version this project targets — continued until June 30, 2011. Sony's underlying DNAS authentication layer, which was shared across many PS2 online titles rather than being specific to Biohazard Outbreak, remained operational for almost a decade afterward and was only shut down on April 4, 2016, alongside the retirement of the last officially supported PS2 online title. This gap of five to nine years between the retirement of the game-specific servers and the retirement of the authentication layer is significant: it means the authentication layer and the game-specific layer are architecturally and organizationally distinct systems, a distinction that directly informed the containerized design adopted in this project.

1.2 Problem

Following these shutdowns, players who wished to experience the cooperative online gameplay of Biohazard Outbreak File #1 and File #2 found themselves with no viable option. The original server architecture relied on proprietary protocols, legacy SSL cipher suites, and authentication mechanisms that were deeply tied to Sony and its matchmaking partner's closed infrastructure.

The core challenges that made restoration difficult include: the DNAS authentication system required specific cryptographic responses that were never publicly documented; the SSL/TLS implementation used cipher suites considered insecure by modern standards and disabled in contemporary software; the game's network protocol operated over raw TCP connections using a binary format specific to the PlayStation 2 hardware; and the matchmaking and session management logic was entirely server-side, meaning the games cannot function in any multiplayer capacity without active server infrastructure.

1.3 Scope

This project focuses on the deployment, configuration, and modernization of a private online server infrastructure for Biohazard Outbreak File #1 and File #2 on the PlayStation 2 platform. The work is built upon an existing open-source server emulation codebase publicly released by the independent developer known as gh0stl1ne (GitLab), released under the GNU Affero General Public License v3.0. Our contributions center on successfully running this software in a modern containerized environment and resolving the compatibility issues that prevented it from operating on current systems. Community resources from the obsrv.org project — whose reconstruction of the Biohazard Outbreak server protocol is documented since 2014, and whose wiki provides client-side (PCSX2/PS2) configuration guidance — were consulted as a supporting reference during testing. The scope includes: deploying a DNAS authentication emulator using Apache HTTP Server compiled with a custom OpenSSL 1.0.2 build; setting up a matchmaking service emulator using PHP and MariaDB; deploying game lobby and session servers for both File #1 and File #2 using Java; containerizing each server component using Docker; resolving legacy compatibility issues; analyzing PS2 network traffic using Wireshark; and establishing secure remote multiplayer connectivity using Tailscale VPN.

1.4 Goals

The primary goals of this project are:

1. Successfully deploy and configure a fully functional private online server infrastructure for Biohazard Outbreak File #1 and File #2.
2. Resolve the compatibility issues preventing the legacy server software from operating on modern environments.
3. Build a custom Apache HTTP Server with OpenSSL 1.0.2 support to enable the weak SSL cipher suites required by the PlayStation 2 hardware.

4. Containerize each server component using Docker to ensure reproducible, isolated, and maintainable deployments.
5. Utilize Wireshark packet analysis to diagnose and resolve network-level connectivity issues.
6. Establish secure remote connectivity between players using Tailscale VPN.
7. Validate the deployment by successfully testing online multiplayer sessions between geographically separated players.

1.5 Solutions

For the DNAS authentication layer, Apache HTTP Server was compiled from source and linked against OpenSSL 1.0.2u, a legacy release that retains support for the weak cipher suites required by the PlayStation 2. This custom build was deployed inside a dedicated Docker container alongside a PHP-based DNAS/matchmaking emulator.

For the matchmaking and account management layer, the service was emulated using PHP 7.4 running under the same Apache instance as the DNAS layer. Deprecated MySQL functions in the original codebase were migrated to the modern MySQLi API. MariaDB was configured to authenticate the application user through the `mysql_native_password` plugin, which is the plugin the legacy MySQL JDBC connector (5.1.49) expects. That older connector version was itself a requirement, not a preference, because the game server source code imports the `com.mysql.jdbc` package, which was removed entirely from Connector/J 8.x in favor of `com.mysql.cj.jdbc`. The connection failures actually encountered during deployment had separate causes — a database user host-scope mismatch and an unsynchronised credential — and are documented in Chapter 5.

For the game server layer, the Biohazard Outbreak server application for both File #1 and File #2 was compiled from Java source code and deployed in dedicated Docker containers, each with its own isolated MariaDB instance. Each server component is defined as an independently deployable Docker unit. Tailscale was configured to create an encrypted peer-to-peer VPN tunnel between players.

1.6 Methodology

This project followed a practical, problem-driven methodology consisting of the following phases:

Environment Setup: Docker containers were designed and built for each server component. A multi-stage Dockerfile was written for the Java-based game servers to compile the source code in a build stage and produce a lean runtime image.

Compatibility Resolution: Each layer of the server stack was tested individually. Errors were diagnosed using Docker logs and Apache error logs. PHP function deprecations, MariaDB authentication failures, and Java database driver incompatibilities were identified and resolved iteratively.

Network Analysis: Wireshark was used to capture and analyze PS2 network traffic at each stage of the connection process, from DNAS authentication through matchmaking and into the game lobby.

Remote Connectivity: Tailscale was deployed to create a secure VPN network between the server host and remote players. Windows Firewall rules were configured to permit game traffic through the VPN interface to the Docker containers.

Validation Testing: The complete system was tested end-to-end with a remote player connecting via Tailscale, successfully completing DNAS authentication, account login, lobby entry, and online multiplayer gameplay.

1.7 Conclusion

This chapter introduced the historical context of the Biohazard Outbreak online servers — including the distinct shutdown timelines of the game-specific and authentication layers — the technical challenges that have prevented their restoration, and the approach taken in this project to overcome those challenges. The following chapters explore related literature and prior work in server emulation and containerization, present the system analysis and design, document the implementation in detail, and report the results of testing and validation.

Chapter 2: Literature Review

2. Literature Review

2.1 Introduction

This chapter presents a review of the existing literature relevant to the key technical areas addressed in this project. Given the interdisciplinary nature of the work — spanning legacy system modernization, containerization technology, and network protocol analysis — the reviewed works are drawn from these three domains. The chapter also discusses how the findings from prior work informed the technical decisions made during this project, and closes with a direct comparison between a known prior attempt at this same server stack and the approach taken here.

2.2 Background

Legacy Systems and the Challenge of Modernization

Legacy systems are software or hardware systems that remain in use despite being built on outdated technology. Piroinen [1] describes such systems as frequently depending on outdated software versions, creating significant challenges from both a security and maintainability perspective. The problem is compounded when the underlying hardware reaches end-of-life, forcing organizations to migrate services to modern platforms without breaking existing functionality.

In the context of this project, the Biohazard Outbreak online server infrastructure represents precisely this type of legacy system. The original servers operated using protocols and cryptographic standards from the early 2000s, including SSL cipher suites that have since been deprecated and removed from modern software libraries. Restoring this infrastructure required bridging a significant compatibility gap between the PlayStation 2's network expectations and the capabilities of modern server environments.

Containerization as a Modernization Strategy

Containerization has emerged as one of the most effective approaches to modernizing legacy software. Watada et al. [2] describe containerization as "extra lightweight, highly portable" and efficient in its use of energy, resources, and storage, offering advantages over traditional virtual machines in terms of boot time, memory footprint, and deployment flexibility. Their comprehensive review demonstrates that containers deliver satisfactory performance across CPU utilization, memory, and network bandwidth, while providing

meaningful isolation between co-located services, though the same review notes that isolation, security, and cross-platform tooling remain open challenges industry-wide.

Pirolainen [1] further demonstrates the practical application of containerization to legacy web services, showing that containerizing aging systems improves maintainability and security by isolating each service along with its dependencies. This isolation means that a service requiring an outdated library version can be containerized without affecting the host system or other services — a property directly applicable to our need to run a custom-compiled Apache HTTP Server with OpenSSL 1.0.2 alongside modern system components.

Network Protocol Analysis

A key challenge in deploying server emulation software for closed, undocumented protocols is understanding the expected communication patterns. Comparetti et al. [3] propose a methodology for automatically inferring protocol specifications from network traces, generating state machines and message format descriptions from captured traffic samples. Their work demonstrates that network traces alone can yield a high-precision approximation of a protocol specification, making packet capture an effective tool for understanding closed protocols.

In this project, Wireshark was used to capture PlayStation 2 network traffic and analyze the sequence of requests made during DNAS authentication, matchmaking, and game lobby connection. This traffic analysis was essential for diagnosing connectivity failures and verifying that each layer of the server stack was responding correctly to the game client.

2.3 Related Work

The Broader Preservation Community

The codebase used in this project did not originate in isolation. The private-server preservation of Biohazard Outbreak traces back to the obsrv.org project, which has documented a working reconstruction of the game's server protocol since 2014 — five years before the gh0st11ne repositories used in this work were first published on GitLab in 2019. Obsrv.org's own documentation describes hosting "a reverse engineered server for Biohazard Outbreak," [7] indicating that the core protocol work — decoding the DNAS authentication sequence and the game's binary session protocol — predates and may be independent of the gh0st11ne codebase. The precise relationship between gh0st11ne's implementation and this earlier community effort could not be established with certainty

from available public records; no claim of original protocol reverse-engineering is made on gh0stl1ne's behalf in this report. What can be stated with confidence is that gh0stl1ne's 2019 release is the earliest publicly available, redistributable source code implementation the authors were able to locate, and it is this implementation — rather than the underlying protocol research itself — that forms the direct basis of this project's deployment work.

The intersection of legacy system emulation, containerization, and game server preservation is a relatively niche area with limited directly comparable academic work. However, several relevant themes emerge from the literature, and a prior community attempt at containerizing this exact server codebase provides a direct point of comparison.

Piironen's [1] migration of legacy research web services to Docker containers shares structural similarities with this project. Both involve software that was no longer actively maintained, dependencies on outdated libraries, and the goal of achieving reproducible deployment on modern infrastructure. The key difference is that our project additionally required compatibility with an external hardware client — the PlayStation 2 — whose network behavior could not be modified, making strict backward compatibility a hard requirement rather than a preference.

Watada et al.'s [2] performance comparison between containers and virtual machines is relevant to the architectural decision to use Docker rather than full virtual machines. Their finding that containers provide comparable performance with significantly lower resource overhead supports the choice of Docker as the deployment platform, particularly given that multiple server components needed to run concurrently on a single host machine.

The work of Comparetti et al. [3] on protocol inference from network traces parallels the network analysis phase of this project. While we did not apply automated protocol inference tools, the underlying principle — using captured network traffic to understand and verify protocol behavior — was central to our troubleshooting methodology.

Comparison with a Prior Deployment Attempt

A previously documented community deployment guide for this same codebase, published by Corbin [8] in March 2023 (hereafter referred to as the "reference deployment"), used a manually configured virtual machine rather than containers, with services managed through a systemd unit and DNS handled by dnsmasq inside the VM. Table 2.1 summarizes the key

differences between that approach and the containerized deployment developed in this project.

Table 2.1. Comparison between the prior VM-based reference deployment and this Docker-based deployment.

Aspect	Reference Deployment (VM-based)	This Project (Docker-based)
Environment	Manually provisioned VM, Debian 11	Docker containers from declarative build files
Service management	systemd unit file	Container entrypoint script
DNS resolution	dnsmasq running inside the VM	PCSX2-side Internal DNS routed over Tailscale
OpenSSL version	1.0.2q	1.0.2u (latest release in the 1.0.2 branch)
Java runtime	OpenJDK 17	OpenJDK 11 (chosen for compatibility with Connector/J 5.1.49)
MySQL/MariaDB driver	Connector/J 8.0.x	Connector/J 5.1.49 (com.mysql.jdbc package required by source code)
Database persistence across updates	Data loss reported in the author's own container attempt	Preserved via a named Docker volume and an idempotent schema-import check
Remote play support	Not covered	Tailscale (WireGuard) VPN tunnel

The most significant divergence is database persistence. The author of the reference deployment also attempted a containerized version of the same stack, but reported that the database server resided inside the container and was destroyed whenever the image was updated, wiping the player database with every update; that attempt was consequently described as unable to replace the virtual-machine guide. This project's container entrypoint script explicitly checks whether the target schema already contains tables before importing bioserver.sql, and stores the data directory in a named Docker volume, so restarting or rebuilding the container does not erase existing player accounts or sessions. This directly satisfies the persistence requirement (NFR5) discussed in Chapter 3.

2.4 Conclusion

The reviewed literature establishes a strong foundation for the technical approach taken in this project. Containerization has been validated as an effective strategy for modernizing legacy software systems. Network traffic analysis has been demonstrated as a reliable method for understanding closed protocol behavior. The direct comparison with a prior

VM-based deployment of the same codebase further shows that containerization, combined with an explicit persistence strategy, addresses a concrete shortcoming reported in that earlier attempt. Together, these bodies of work support the design decisions made in this project and provide academic grounding for the practical solutions implemented.

Chapter 3: System Analysis and Design

Figure 1. Wireshark packet capture of the PS2 DNAS authentication exchange on TCP port 443, showing the Client Hello and the TLSv1 handshake.

This analysis revealed the following key requirements: the game first contacts the DNAS server at gate1.jp.dnas.playstation.org on port 443 using legacy SSL ciphers; after DNAS authentication, the game contacts a matchmaking server at www01.kddi-mmbb.jp on port 443; following successful login, the game connects to the lobby server on port 8200 (File #2) or port 8300 (File #1); and game sessions are handled on a separate port: 8590 (File #2) or 8690 (File #1). Figure 2 illustrates this full sequence.

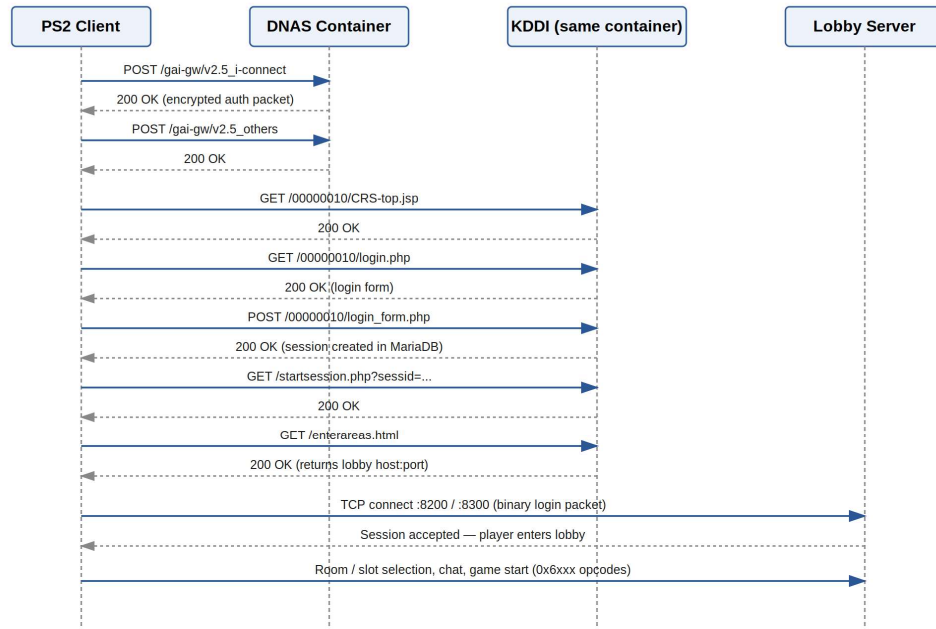


Figure 2. Sequence diagram of the PS2 client connection process, from DNAS authentication through lobby entry.

3.3 System Requirements

3.3.1 Functional Requirements

Table 3.1. Functional requirements of the server infrastructure.

#	Requirement
FR1	The system shall authenticate PlayStation 2 clients through a DNAS emulation server supporting legacy SSL cipher suites
FR2	The system shall provide a web-based login and account creation interface accessible from the PS2 browser
FR3	The system shall store and retrieve player accounts and session data using a relational database

FR4	The system shall provide lobby server functionality for Biohazard Outbreak File #1 on port 8300
FR5	The system shall provide lobby server functionality for Biohazard Outbreak File #2 on port 8200
FR6	The system shall handle game session connections for File #1 on port 8690 and File #2 on port 8590
FR7	The system shall redirect PlayStation 2 DNS queries for the Sony/matchmaking domains to the server IP
FR8	The system shall support remote player connectivity through an encrypted VPN tunnel

3.3.2 Non-functional Requirements

Table 3.2. Non-functional requirements and their associated constraints.

#	Requirement
NFR1	Compatibility: The system shall present an SSL cipher suite acceptable to the PlayStation 2 client, negotiated as EDH-RSA-DES-CBC3-SHA / TLS_DHE_RSA_WITH_3DES_EDE_CBC_SHA (the cipher configuration excludes AES, SEED, and Camellia families to force selection of this legacy cipher)
NFR2	Isolation: Server components shall be separated into distinct Docker containers by role (authentication/matchmaking vs. game logic) to prevent dependency conflicts between incompatible library versions
NFR3	Reproducibility: Each server component shall be independently deployable and reproducible from its own Docker build/Compose definition, without manual host configuration of that component; inter-component network wiring is performed as a separate step (see Chapter 6, Limitation 3)
NFR4	Security: Remote connectivity shall be encrypted using a WireGuard-based VPN (Tailscale)
NFR5	Persistence: Player accounts and session data shall persist across container restarts and rebuilds, via a Docker-managed volume and an idempotent schema-import check
NFR6	Compatibility: The HTTP server shall downgrade to HTTP/1.0 for requests matching the PS2 browser signature, since the PS2 client does not support HTTP/1.1 keep-alive semantics

Note on NFR2: within each game-server container, the database engine, web server, and Java runtime run as separate processes inside a single container rather than one process per container. This is a deliberate, documented deviation from strict container design orthodoxy, discussed further as a limitation in Chapter 6, made necessary because the original server code assumes an always-available local database connection.

3.3.3 Use Cases

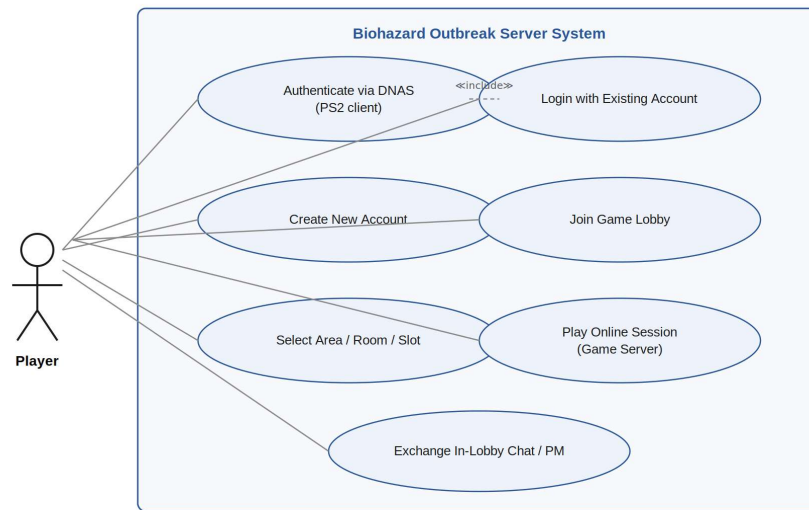


Figure 3. Use case diagram for player interaction with the server system.

Use Case 1: Player Authentication via DNAS

Table 3.3. Use case 1 — Player authentication via DNAS.

Field	Description
Actor	PlayStation 2 game client
Precondition	Game is launched and network is configured
Main Flow	Game contacts DNAS server → Server verifies request using pre-computed response packets → Server returns authentication confirmation
Postcondition	Game proceeds to matchmaking
Exception	If the response packet is not found, the server returns error.raw and the game displays error code 626

Use Case 2: Player Login

Table 3.4. Use case 2 — Player login.

Field	Description
Actor	Player
Precondition	DNAS authentication successful
Main Flow	Game loads login page → Player enters ID and password → Server validates credentials against database → Session created → Player redirected to lobby selection
Postcondition	Player session established in database
Exception	Invalid credentials return login failed message

Use Case 3: Create New Account

Table 3.5. Use case 3 — Create new account.

Field	Description
Actor	Player
Precondition	DNAS authentication successful
Main Flow	Player enters new ID and password → Server checks if username exists → Account created in database → Session established → Player enters lobby
Postcondition	New account stored in MariaDB
Exception	Username already exists returns error message

Use Case 4: Join Game Lobby

Table 3.6. Use case 4 — Join game lobby.

Field	Description
Actor	Player
Precondition	Player successfully logged in
Main Flow	Game receives lobby server address from enterareas.html → Game connects to Java lobby server on port 8200/8300 → Player appears in lobby list
Postcondition	Player connected to lobby server
Exception	Connection failure returns communication error

3.3.4 Context Diagram

Figure 4 illustrates the overall system architecture and the interactions between all components.

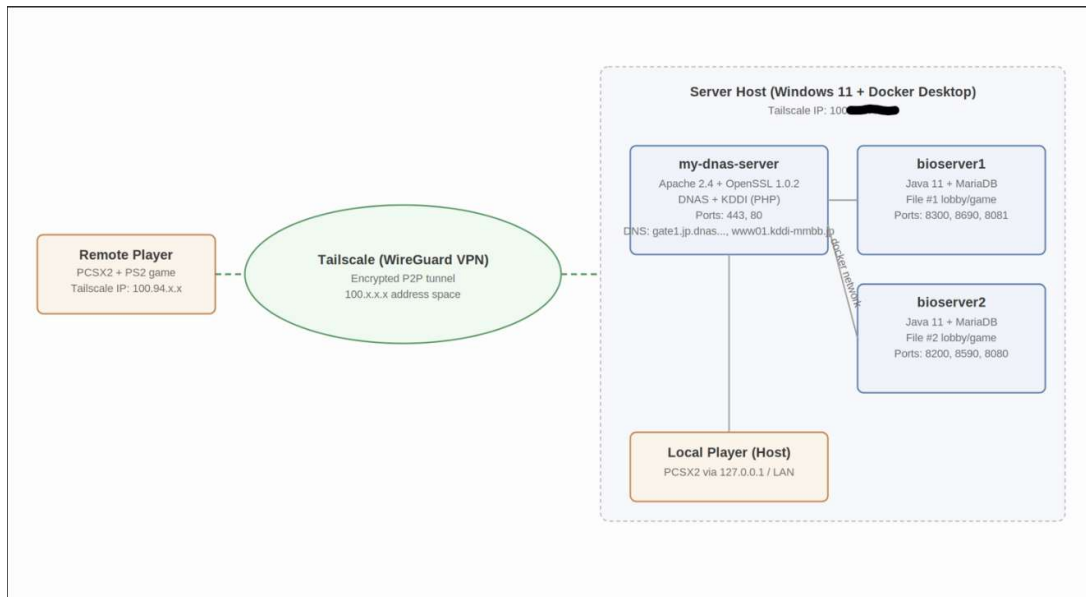


Figure 4. Context / network topology diagram showing the DNAS container, per-title game server containers, Docker networking, and the Tailscale VPN tunnel to remote players.

The system consists of three primary components. The DNAS container handles all HTTPS traffic on port 443 from the PS2 client; it runs Apache HTTP Server compiled with OpenSSL 1.0.2u and serves both the DNAS authentication responses and the matchmaking PHP pages. Each Bioserver container (one per game title) handles TCP connections on the lobby and game ports; it runs the Java server application and contains its own MariaDB instance for player data storage. Tailscale VPN provides the encrypted network tunnel that allows remote players to connect to the server as if they were on the same local network.

3.4 The Prototype of System Interface

The following figures show the deployed system as it appears to an administrator and to a player. Together they document the running container stack, the client-side DNS configuration required on the PS2 emulator, the login interface rendered inside the game, and the server-side log evidence that the authentication and matchmaking requests were served successfully.

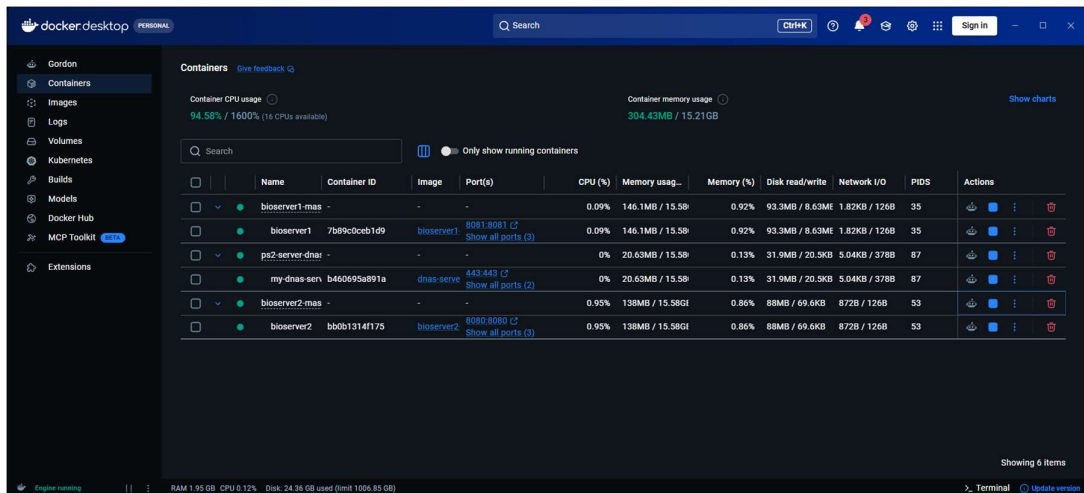


Figure 5. Docker Desktop showing all server containers running: my-dnas-server, bioserver1, and bioserver2.

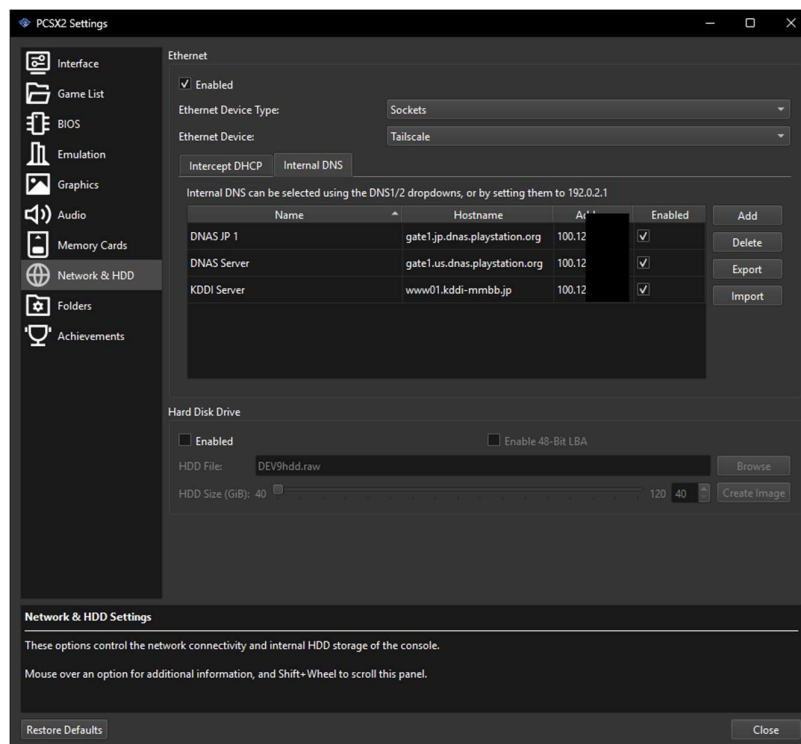


Figure 6. PCSX2 Network settings with the Internal DNS entries mapped to the Tailscale address of the server host.

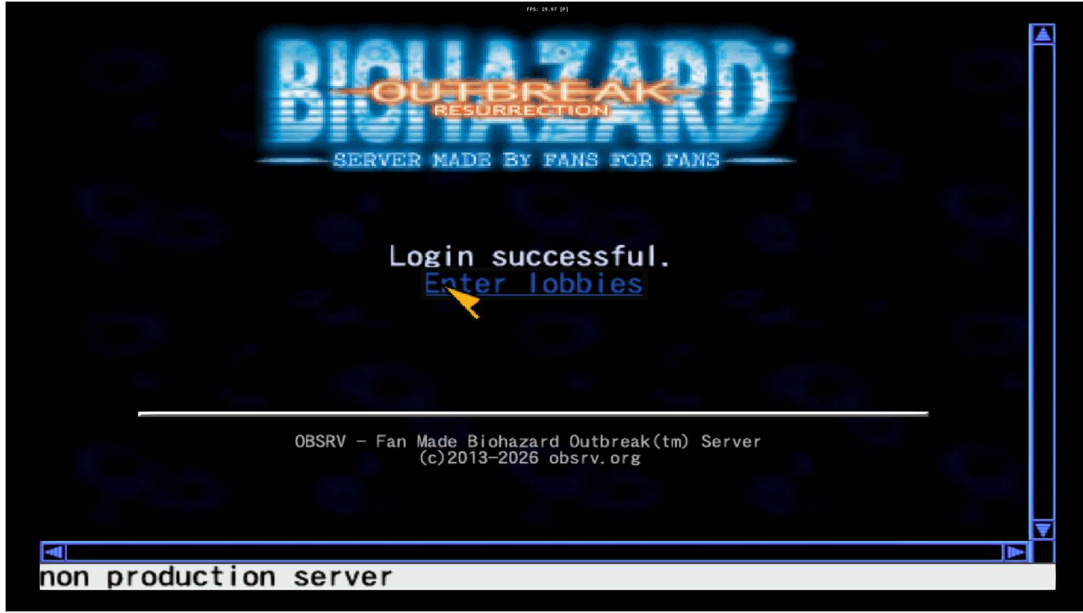


Figure 7. Biohazard Outbreak login result screen in PCSX2 confirming a successful account login.

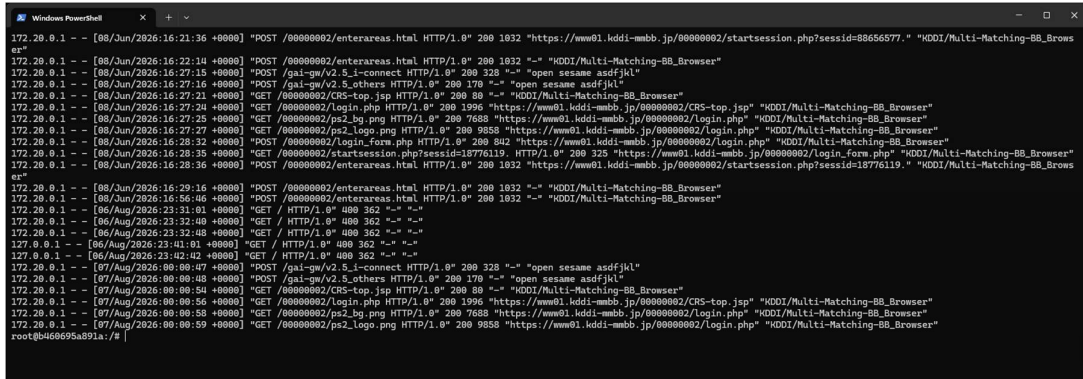


Figure 8. Apache access log showing successful DNAs and matchmaking requests returning HTTP 200.

3.5 Conclusion

This chapter presented the system requirements derived from direct network traffic analysis of PlayStation 2 communication patterns. The functional requirements established the need for a multi-layer server infrastructure covering DNAs authentication, web-based matchmaking, and Java-based lobby and game session management. The non-functional requirements highlighted the critical compatibility constraints imposed by the PlayStation 2 hardware. The context and sequence diagrams illustrated how these components interact through Docker networking and Tailscale VPN to deliver a complete online gaming experience.

Chapter 4: Application

4. Application

4.1 Introduction

This chapter documents the implementation of the private online server infrastructure for Biohazard Outbreak File #1 and File #2. It covers the tools and technologies used, the deployment architecture, the user-facing interface, the database design, and the containerization structure. All components were deployed on a Windows 11 host machine using Docker Desktop, with remote connectivity provided through Tailscale VPN.

4.2 Tools and Programming Languages

Table 4.1. Tools, technologies, and versions used in the deployment.

Tool / Technology	Version	Purpose
Docker Engine	29.5.2	Container management and deployment
Docker Compose	v5.1.4	Per-service container definition
Apache HTTP Server	2.4.56 (compiled from source for the DNAS container)	Web server for DNAS and matchmaking emulation
OpenSSL	1.0.2u	Legacy SSL cipher support for PS2
PHP	7.4	Server-side scripting for matchmaking
MariaDB	10.5.29 (Debian Bullseye repository)	Relational database for player data
Java (OpenJDK)	11	Game lobby and session server runtime, paired with Connector/J 5.1.49
MySQL Connector/J	5.1.49	Legacy JDBC driver — required because the game server source imports <code>com.mysql.jdbc</code> , a package removed in Connector/J 8.x
Tailscale	1.98	WireGuard-based VPN for remote connectivity
Wireshark	4.6.5	Network traffic capture and analysis
PCSX2	2.6.3	PlayStation 2 emulator used for testing
Debian	Bullseye (11)	Base operating system for all containers
Bash	5.x	Entrypoint scripting for container initialization

4.3 Implementation of User Interface

The user interface in this system is the web-based login page served by the DNAS container and displayed within the PlayStation 2 game client. This interface is not a traditional web application — it is rendered by the PS2's built-in browser, which supports only basic HTML and HTTP/1.0.

Step 1 — DNAS Authentication

When the game launches and attempts to connect online, it first contacts the DNAS server at `gate1.jp.dnas.playstation.org`. The server responds with a pre-computed authentication packet, allowing the game to proceed.

The corresponding packet capture is shown in Figure 1.

Step 2 — Login Page

After DNAS authentication, the game loads the login page from the matchmaking server. The page presents two options: login with an existing account, or create a new account.

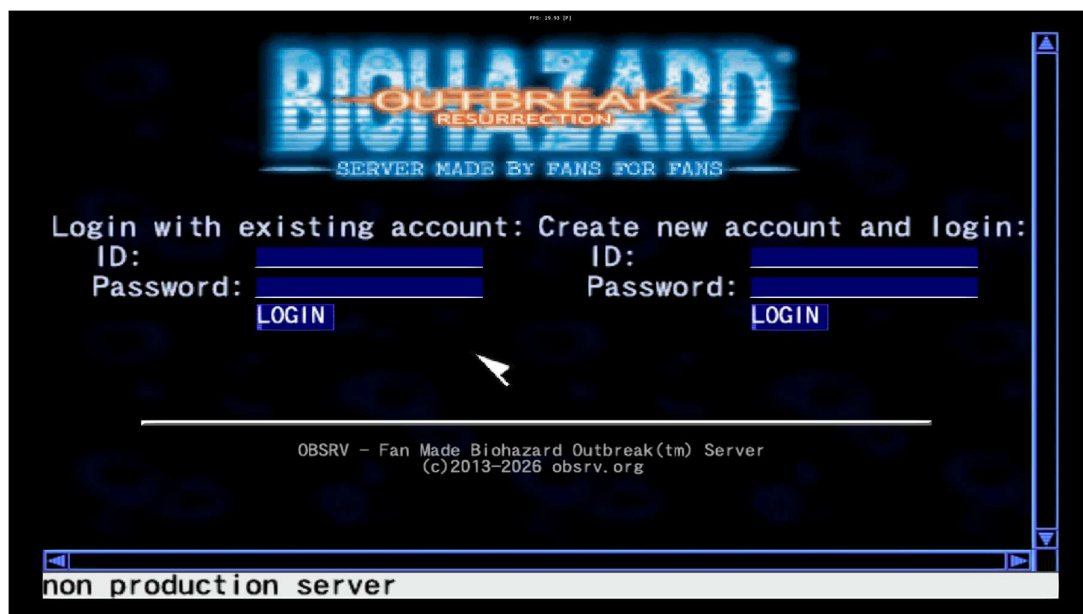


Figure 9. Biohazard Outbreak login page in PCSX2 showing the ID and Password fields for both existing and new accounts.

Step 3 — Successful Login

Upon successful authentication, the server creates a session in the database and returns a confirmation page with a link to enter the game lobbies.

The resulting confirmation screen is shown in Figure 7.

Step 4 — Lobby Entry

The game connects to the Java lobby server using the address provided in `enterareas.html`. The player appears in the lobby, where existing game rooms can be browsed and a new room can be created with its own session parameters.



Figure 10. Room creation screen served by the Java lobby server, showing the configurable session parameters: player count, wait limit, difficulty, friendly fire, scenario, and character.

Step 5 — Online Multiplayer

Players in the same lobby can create or join game sessions, which are handled by the Java game server on a separate port.



Figure 11. Two players connected to the same online session, shown at the character selection stage.

4.4 Database Application

The database used in this project is MariaDB, an open-source relational database management system. Each Bioserver container maintains its own isolated MariaDB instance. Figure 12 shows the entity-relationship structure of the schema, which contains four tables.

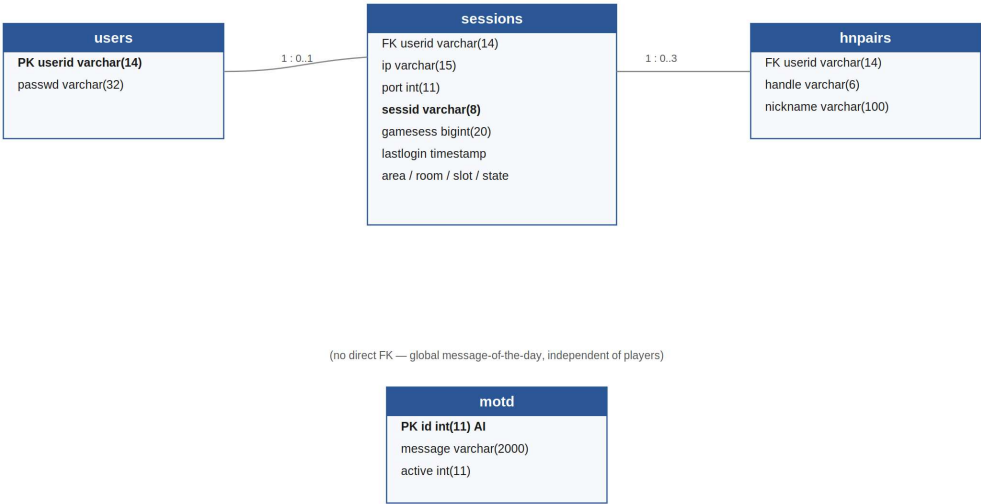


Figure 12. Entity-relationship diagram of the player database schema.

sessions table — stores active player sessions:

Table 4.2. Structure of the sessions table.

Column	Type	Description
userid	varchar(14)	Player username
ip	varchar(15)	Player IP address
port	int(11)	Player connection port
sessid	varchar(8)	Unique session identifier
gamesess	bigint(20)	Game session number
lastlogin	timestamp	Last login time
area	int(11)	Current area (-1 if none)
room	int(11)	Current room
slot	int(11)	Current slot
state	int(11)	Connection state

users table — stores player accounts:

Table 4.3. Structure of the users table.

Column	Type	Description
userid	varchar(14)	Player username (Primary Key)
passwd	varchar(32)	Player password

Note: the passwd field is stored in plaintext-compatible form as defined by the original 2004-era protocol, and was left unmodified to preserve client compatibility. This is documented as a limitation in Chapter 6.

hnpairs table — stores player handles and nicknames:

Table 4.4. Structure of the hnpairs table.

Column	Type	Description
userid	varchar(14)	Player username
handle	varchar(6)	In-game handle
nickname	varchar(100)	Display nickname (UTF-8)

motd table — stores message of the day:

Table 4.5. Structure of the motd table.

Column	Type	Description
id	int(11)	Auto-increment ID
message	varchar(2000)	Message content
active	int(11)	Active flag (1 = displayed)


```

Windows PowerShell
PS C:\Users\lncc7> docker exec -it bioserver1 mariadb -u root -p -e "SHOW DATABASES;"
Enter password:
+-----+
| Database |
+-----+
| bioserver |
| information_schema |
| mysql |
| performance_schema |
+-----+

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug bioserver1
Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\lncc7> docker exec -it bioserver1 mariadb -u root -p -e "SHOW TABLES FROM bioserver;"
Enter password:
+-----+
| Tables_in_bioserver |
+-----+
| hnpairs |
| motd |
| sessions |
| users |
+-----+

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug bioserver1
Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\lncc7>

```

Figure 13. MariaDB databases and tables inside the bioserver1 container, listed via docker exec.

4.5 Packages and Containers

The system is composed of three Docker containers, each defined by its own Dockerfile and Compose file rather than a single unified orchestration file (see NFR3 in Chapter 3).

Container 1: my-dnas-server

Table 4.6. Composition of Container 1 (my-dnas-server).

Component	Detail
Web Server	Apache 2.4.56 compiled from source against OpenSSL 1.0.2u
Build stages	Single-stage build; build toolchain and source archives remain in the final image (see Limitations, Chapter 6)
SSL Cipher	EDH-RSA-DES-CBC3-SHA, forced via SSLCipherSuite DHE:!DSS:!AES:!SEED:!CAMELLIA:!TLSv1.2
Ports	443 (HTTPS), 80 (HTTP)
PHP Scripts	connect.php, login_form.php, db_cred.php, startsession.php, enterareas.html
Served hostnames	gate1.jp.dnas.playstation.org, www01.kddi-mmabb.jp (resolved via PCSX2 Internal DNS, not a DNS service inside the container)

Container 2: bioserver2

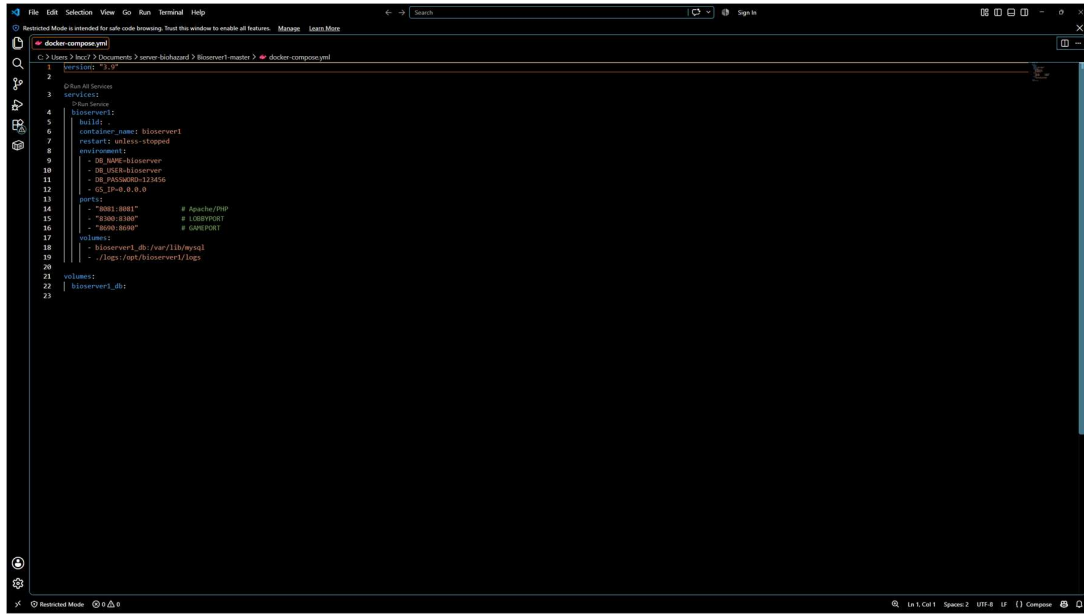
Table 4.7. Composition of Container 2 (bioserver2).

Component	Detail
Build stages	Multi-stage: builder stage compiles Java source; runtime stage copies only the compiled JAR and dependencies
Runtime	OpenJDK 11
Database	MariaDB (bioserver2), isolated volume
Lobby Port	8200
Game Port	8590
Web Port	8080 (Apache/PHP, standard apt package — not the custom OpenSSL build)
JAR	BiohazardOutbreak2Server.jar

Container 3: bioserver1

Table 4.8. Composition of Container 3 (bioserver1).

Component	Detail
Build stages	Multi-stage: builder stage compiles Java source; runtime stage copies only the compiled JAR and dependencies
Runtime	OpenJDK 11
Database	MariaDB (bioserver), isolated volume
Lobby Port	8300
Game Port	8690
Web Port	8081 (Apache/PHP, standard apt package)
JAR	BiohazardOutbreak1Server.jar
Patch asset	patch_101.raw copied into the image; consumed by the original client-update feature of File #1 (absent in File #2)



```
1 version: '3.9'
2
3 services:
4   bioserver1:
5     build:
6       context: ./src
7     environment:
8       - DB_NAME=bioserver
9       - DB_USER=bioserver
10      - DB_PASSWORD=123456
11      - DB_HOST=0.0.0.0
12     ports:
13       - "8081:8081" # Application Port
14       - "8080:8080" # LogPort
15       - "8090:8090" # GamePort
16     volumes:
17       - bioserver1_db:/var/lib/mysql
18       - ./logs:/opt/bioserver1/logs
19     volumes:
20       bioserver1_db:
21       bioserver1_db:
```

Figure 14. *docker-compose.yml* for bioserver1, showing the service definition, environment variables, and port mappings.

Multi-Stage Dockerfile (bioserver1 / bioserver2)

The Bioserver containers were built using a multi-stage Dockerfile. The first stage (builder) installs the JDK, compiles the Java source code, and packages it into a JAR file. The second stage (runtime) copies only the compiled JAR, the legacy MySQL connector, and the necessary runtime dependencies — the JDK itself is not present in the final image, reducing its size relative to a single-stage equivalent. The DNAS container, by contrast, is currently built as a single stage; this asymmetry is noted explicitly in Chapter 6 as a limitation and a target for future cleanup.

```
FROM debian:bullseye-slim AS builder

# Stage 1: Build JAR
FROM debian:bullseye-slim AS builder

RUN apt-get update && apt-get install -y \
    wget \
    curl \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /build

COPY bioserver1-master/bioserver/ ./bioserver/

# MySQL connector 5.1.49 (required, uses com.mysql.jdbc old API)
RUN apt-get install http://repo.mysql.com/mysql-connector-java-5.1.49.jar \
    && mv mysql-connector.jar \
    && rm -rf /var/lib/apt/lists/*

# Compile all .java files
RUN mkdir -p out && \
    find bioserver/ -name '*.java' > sources.txt && \
    javac -cp mysql-connector.jar \
    @sources.txt

# Package into JAR
RUN echo "Main-Class: bioserver.ServerMain" > manifest.txt && \
    echo "Class-Path: mysql-connector.jar" > manifest.txt && \
    echo "" > manifest.txt && \
    jar -fm bioserver/bioserver.jar manifest.txt -C out .

# Stage 2: Runtime Image
FROM debian:bullseye-slim

ENV DEBIAN_FRONTEND=noninteractive \
    DB_USER=bioserver \
    DB_PASS=bioserver \
    DB_NAME=bioserver \
    DB_HOST=localhost \
    DB_PORT=3306

RUN apt-get update && apt-get install -y \
    wget \
    curl \
    mariadb-server \
    mariadb \
    php \
    libapache2-mod-php \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /opt/bioserver

# Copy compiled JAR & connector from builder
COPY --from=builder /build/bioserver/bioserver.jar ./
COPY --from=builder /build/mysql-connector.jar ./

# Copy config & database schema
COPY bioserver1-master/bioserver/config.properties ./
COPY bioserver1-master/initdb/bioserver.sql /docker-entrypoint-initdb/bioserver.sql

# Copy some files (from files)
COPY bioserver1-master/www/ /var/www/html/
COPY bioserver1-master/entrypoint.sh ./

# Configure Apache on port 8081 (8089 if you have bioserver2)
RUN sed -i 's/Listen 80/Listen 8081/' /etc/apache2/ports.conf && \
    sed -i 's/Listen 80/Listen 8081/' /etc/apache2/sites-enabled/000-default.conf

# Ports
EXPOSE 8081 8089 8080
# 8081 - bioserver
# 8089 - bioserver2
# 8080 - connector
```

Figure 15. Dockerfile showing the two-stage build: the JAR builder stage and the slim runtime image stage.

Modifications to the Upstream Source

The upstream repositories were written for a single-machine deployment and required three categories of change before they could run as separate containers. These are recorded here because they are modifications to AGPL-3.0 licensed source, not merely configuration.

1. Database provisioning moved out of the schema file. The upstream bioserver.sql created the database, the application user, and its grants inline, with the password written as a literal placeholder. A static SQL file cannot read Docker environment variables, so those three statements were removed and reimplemented in docker-entrypoint.sh using `${DB_USER}`, `${DB_PASSWORD}`, and `${DB_NAME}`. CREATE USER IF NOT EXISTS was used in place of the upstream CREATE USER so that container restarts remain idempotent. The file now contains only the schema definition, allowing the same image to be built for either title by changing environment variables alone.

2. Database host changed from loopback to container name. The upstream db_cred.php connected to localhost, which is correct when PHP and the database share a machine. In this deployment the matchmaking pages run in the DNAS container while the databases run inside the Bioserver containers, so the host was changed to the container name (bioserver1 or bioserver2). This resolves only because the DNAS container is attached to both Bioserver Compose networks, which provide internal name resolution. The same file

was also migrated from the `mysql_*` functions, removed in PHP 7.0, to `mysqli_connect`, which accepts the database name as a fourth argument and replaces the separate `mysql_select_db` call.

3. Per-title matchmaking directories published under the DNAS document root. The game requests its matchmaking pages by a fixed address identifier: File #1 requests `/00000002/` and File #2 requests `/00000010/`, as visible in the server access log (Figure 8). These paths are compiled into the client and cannot be changed. Because only the DNAS container runs the custom OpenSSL 1.0.2u build required to serve the PlayStation 2 on port 443, the matchmaking pages for both titles were published there, each in its own directory with a `db_cred.php` pointing at that title's database. The upstream DNASrep project ships only the Sony authentication gateways (`gai-gw` and its regional siblings); these two directories were added by this project and are what allow a single DNAS container to serve both titles.

Container Startup Sequence

Each Bioserver container follows a defined startup sequence managed by `docker-entrypoint.sh`: (1) start MariaDB and wait for it to accept connections; (2) create the database, application user, and grant privileges; (3) check whether the target schema already contains tables, and only import `bioserver.sql` if it does not — this idempotency check is what allows the container to be restarted or rebuilt without erasing existing player accounts and sessions (see NFR5, and the comparison with the reference deployment in Chapter 2); (4) reconfigure MariaDB to bind to all network interfaces and restart it; (5) start Apache; (6) launch the Java game server JAR.

```
Windows PowerShell
- (c) 2013-2019 obsrv.org
- no23@deathless.net

Thu Jun 04 16:26:33 UTC 2026 server started
==> Starting MariaDB...
Starting MariaDB database server: mariadb.
mysqld is alive
==> Setting up database...
Database already initialized, skipping.
==> Starting Apache...
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 172.19.0.3. Set the 'ServerName' directive globally to suppress this message
Starting Apache httpd web server: apache2.
==> Starting Biohazard Outbreak 2 Server...
Thu Aug 06 21:36:16 UTC 2026 Gameserver IP: /192.168.100.7

- farmade server for -
- Biohazard Outbreak File #2 -
- -
- (c) 2013-2019 obsrv.org -
- no23@deathless.net -

Thu Aug 06 21:36:17 UTC 2026 server started

What's next:
View and search logs for all containers in one place
with Docker Desktop's Logs view. docker-desktop://dashboard/logs
PS C:\Users\lncc7> |
```

Figure 16. Startup log of bioserver2 showing MariaDB, Apache, and the Java game server initialising, including the schema-already-initialised skip message.

VPN Connectivity

Tailscale was configured to create a secure peer-to-peer VPN tunnel between the server host and remote players. Each device receives a Tailscale IP in the 100.x.x.x range. Remote players configure their PCSX2 Internal DNS to point to the server's Tailscale IP instead of the local network IP. One further step is required on the server side: the lobby server does not carry game traffic itself, it advertises the address of the session server to the client. That address is read from the `gs_ip` entry in `config.properties`, which was therefore set to the host's Tailscale address. Had it been left at a loopback or LAN address, a remote client would have been directed back to its own machine after leaving the lobby, and the session would never have started.

4.6 Conclusion

This chapter documented the complete implementation of the private Biohazard Outbreak server infrastructure. The system was successfully deployed across three Docker containers, each serving a distinct role in the connection chain from DNAS authentication through game session management. Compatibility challenges were addressed through custom compilation of Apache with legacy OpenSSL support, migration of PHP database functions, and configuration of the MariaDB application user to authenticate through the plugin expected by the legacy Java MySQL connector. Remote multiplayer connectivity

was achieved through Tailscale VPN, enabling players at different network locations to connect to the server securely.

Chapter 5: Test and Results

5. Test and Results

5.1 Introduction

This chapter presents the testing methodology and results for the deployed Biohazard Outbreak private server infrastructure. Testing was conducted across three levels: unit testing of individual server components, integration testing of the complete connection chain, and user acceptance testing through actual online multiplayer sessions. Rather than presenting only the final passing state, this chapter also documents the specific failures encountered during integration and acceptance testing and how each was diagnosed and resolved, since those iterations were a substantial part of the engineering effort.

5.2 Testing System

5.2.1 Unit Testing

Unit testing was performed on each server component independently to verify correct operation before integration.

DNAS Container Test

The DNAS authentication service functionality was verified end-to-end by initiating a connection from the PCSX2 emulator running the target title. Successful processing of the legacy SSL/TLS authentication request was confirmed via container runtime logs (docker logs my-dnas-server) and network packet capture, validating that the server correctly receives, decrypts, and responds to authentic PS2 client payloads.

An excerpt from the resulting DNAS container log is shown in Figure 17.

Apache SSL Test

The Apache server's SSL configuration was verified by checking that the legacy cipher suite required by the PS2 was available and active in the negotiated connection log.

```
root@b0b6605a91a: /# grep -i "EDH" /opt/apache/logs/dnas_error.log
[Sat May 30 00:33:57.115652 2020] [ssl:debug] [pid 27:t/td 124921524582140] ssl_engine_kernel.c(2254): [client 172.18.0.1:55966] AH02801: Protocol: TLSv1, Cipher: EDH-RSA-DES-CBC3-SHA (112/168 bits)
[Sat May 30 00:33:58.123092 2020] [ssl:debug] [pid 27:t/td 124921527790736] ssl_engine_kernel.c(2254): [client 172.18.0.1:56078] AH02801: Protocol: TLSv1, Cipher: EDH-RSA-DES-CBC3-SHA (112/168 bits)
[Sat May 30 00:34:03.088206 2020] [ssl:debug] [pid 197:t/td 124921526582140] ssl_engine_kernel.c(2254): [client 172.18.0.1:56092] AH02801: Protocol: TLSv1, Cipher: EDH-RSA-DES-CBC3-SHA (112/168 bits)
[Sat May 30 01:02:55.869090 2020] [ssl:debug] [pid 26:t/td 131456852988316] ssl_engine_kernel.c(2254): [client 172.18.0.1:37938] AH02801: Protocol: TLSv1, Cipher: EDH-RSA-DES-CBC3-SHA (112/168 bits)
[Sat May 30 01:02:59.868676 2020] [ssl:debug] [pid 26:t/td 131456853134080] ssl_engine_kernel.c(2254): [client 172.18.0.1:51970] AH02801: Protocol: TLSv1, Cipher: EDH-RSA-DES-CBC3-SHA (112/168 bits)
[Sat May 30 01:03:04.124013 2020] [ssl:debug] [pid 26:t/td 131456851828980] ssl_engine_kernel.c(2254): [client 172.18.0.1:51982] AH02801: Protocol: TLSv1, Cipher: EDH-RSA-DES-CBC3-SHA (112/168 bits)
[Sat May 30 19:09:33.211828 2020] [ssl:debug] [pid 26:t/td 133866294811648] ssl_engine_kernel.c(2254): [client 172.18.0.1:58078] AH02801: Protocol: TLSv1, Cipher: EDH-RSA-DES-CBC3-SHA (112/168 bits)
[Sat May 30 19:09:34.209588 2020] [ssl:debug] [pid 197:t/td 133866294811648] ssl_engine_kernel.c(2254): [client 172.18.0.1:58096] AH02801: Protocol: TLSv1, Cipher: EDH-RSA-DES-CBC3-SHA (112/168 bits)
[Sat May 30 19:09:39.151865 2020] [ssl:debug] [pid 197:t/td 133866293379360] ssl_engine_kernel.c(2254): [client 172.18.0.1:58096] AH02801: Protocol: TLSv1, Cipher: EDH-RSA-DES-CBC3-SHA (112/168 bits)
[Sat May 30 19:10:32.865948 2020] [ssl:debug] [pid 197:t/td 13386618689652] ssl_engine_kernel.c(2254): [client 172.18.0.1:37886] AH02801: Protocol: TLSv1, Cipher: EDH-RSA-DES-CBC3-SHA (112/168 bits)
[Sat May 30 19:10:40.992204 2020] [ssl:debug] [pid 197:t/td 133866169804560] ssl_engine_kernel.c(2254): [client 172.18.0.1:38164] AH02801: Protocol: TLSv1, Cipher: EDH-RSA-DES-CBC3-SHA (112/168 bits)
[Sat May 30 19:36:57.128614 2020] [ssl:debug] [pid 26:t/td 133866283733680] ssl_engine_kernel.c(2254): [client 172.18.0.1:58628] AH02801: Protocol: TLSv1, Cipher: EDH-RSA-DES-CBC3-SHA (112/168 bits)
[Sat May 30 19:36:59.668017 2020] [ssl:debug] [pid 197:t/td 133866074807152] ssl_engine_kernel.c(2254): [client 172.18.0.1:58628] AH02801: Protocol: TLSv1, Cipher: EDH-RSA-DES-CBC3-SHA (112/168 bits)
[Sat May 30 19:37:01.399724 2020] [ssl:debug] [pid 197:t/td 133866049733680] ssl_engine_kernel.c(2254): [client 172.18.0.1:66996] AH02801: Protocol: TLSv1, Cipher: EDH-RSA-DES-CBC3-SHA (112/168 bits)
[Sat May 30 19:37:02.918326 2020] [ssl:debug] [pid 197:t/td 133866041326336] ssl_engine_kernel.c(2254): [client 172.18.0.1:66992] AH02801: Protocol: TLSv1, Cipher: EDH-RSA-DES-CBC3-SHA (112/168 bits)
[Sat May 30 19:43:51.226241 2020] [ssl:debug] [pid 197:t/td 133866024404928] ssl_engine_kernel.c(2254): [client 172.18.0.1:37972] AH02801: Protocol: TLSv1, Cipher: EDH-RSA-DES-CBC3-SHA (112/168 bits)
[Sat May 30 19:44:07.828867 2020] [ssl:debug] [pid 197:t/td 133866129633360] ssl_engine_kernel.c(2254): [client 172.18.0.1:51388] AH02801: Protocol: TLSv1, Cipher: EDH-RSA-DES-CBC3-SHA (112/168 bits)
[Sat May 30 19:44:10.369028 2020] [ssl:debug] [pid 197:t/td 133866112848128] ssl_engine_kernel.c(2254): [client 172.18.0.1:51388] AH02801: Protocol: TLSv1, Cipher: EDH-RSA-DES-CBC3-SHA (112/168 bits)
[Sat May 30 19:44:11.979793 2020] [ssl:debug] [pid 26:t/td 13386618689652] ssl_engine_kernel.c(2254): [client 172.18.0.1:52968] AH02801: Protocol: TLSv1, Cipher: EDH-RSA-DES-CBC3-SHA (112/168 bits)
[Sat May 30 19:44:13.498776 2020] [ssl:debug] [pid 197:t/td 133866090862720] ssl_engine_kernel.c(2254): [client 172.18.0.1:53882] AH02801: Protocol: TLSv1, Cipher: EDH-RSA-DES-CBC3-SHA (112/168 bits)
[Sat May 30 20:10:31.393793 2020] [ssl:debug] [pid 197:t/td 133866081153228] ssl_engine_kernel.c(2254): [client 172.18.0.1:37928] AH02801: Protocol: TLSv1, Cipher: EDH-RSA-DES-CBC3-SHA (112/168 bits)
[Sat May 30 20:11:23.666083 2020] [ssl:debug] [pid 197:t/td 133866194369920] ssl_engine_kernel.c(2254): [client 172.18.0.1:41838] AH02801: Protocol: TLSv1, Cipher: EDH-RSA-DES-CBC3-SHA (112/168 bits)
[Sat May 30 20:11:26.183944 2020] [ssl:debug] [pid 197:t/td 13386621768864] ssl_engine_kernel.c(2254): [client 172.18.0.1:41822] AH02801: Protocol: TLSv1, Cipher: EDH-RSA-DES-CBC3-SHA (112/168 bits)
[Sat May 30 20:11:27.816112 2020] [ssl:debug] [pid 197:t/td 133866194369920] ssl_engine_kernel.c(2254): [client 172.18.0.1:41824] AH02801: Protocol: TLSv1, Cipher: EDH-RSA-DES-CBC3-SHA (112/168 bits)
[Sat May 30 20:11:29.392818 2020] [ssl:debug] [pid 197:t/td 133866178197280] ssl_engine_kernel.c(2254): [client 172.18.0.1:41830] AH02801: Protocol: TLSv1, Cipher: EDH-RSA-DES-CBC3-SHA (112/168 bits)
[Sat May 30 20:51:35.140337 2020] [ssl:debug] [pid 197:t/td 133866161411840] ssl_engine_kernel.c(2254): [client 172.18.0.1:55852] AH02801: Protocol: TLSv1, Cipher: EDH-RSA-DES-CBC3-SHA (112/168 bits)
[Sat May 30 20:52:36.542394 2020] [ssl:debug] [pid 26:t/td 133866169804560] ssl_engine_kernel.c(2254): [client 172.18.0.1:55202] AH02801: Protocol: TLSv1, Cipher: EDH-RSA-DES-CBC3-SHA (112/168 bits)
[Sat May 30 20:52:39.958694 2020] [ssl:debug] [pid 197:t/td 133866666504480] ssl_engine_kernel.c(2254): [client 172.18.0.1:55286] AH02801: Protocol: TLSv1, Cipher: EDH-RSA-DES-CBC3-SHA (112/168 bits)
[Sat May 30 20:52:40.659615 2020] [ssl:debug] [pid 197:t/td 133866681117440] ssl_engine_kernel.c(2254): [client 172.18.0.1:58118] AH02801: Protocol: TLSv1, Cipher: EDH-RSA-DES-CBC3-SHA (112/168 bits)
```

Figure 17. DNAS SSL debug log confirming the negotiated legacy cipher suite EDH-RSA-DES-CBC3-SHA over TLSv1.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\lnc7> wsl wget http://127.0.0.1/
--2026-08-07 02:32:48-- http://127.0.0.1/
Connecting to 127.0.0.1:80... connected.
HTTP request sent, awaiting response... 400 Bad Request
2026-08-07 02:32:48 ERROR 400: Bad Request.

PS C:\Users\lnc7> wsl wget https://127.0.0.1/
--2026-08-07 02:33:50-- https://127.0.0.1/
Connecting to 127.0.0.1:443... connected.
GnuTLS: A TLS fatal alert has been received.
GnuTLS: received alert [40]: Handshake failed
Unable to establish SSL connection.
PS C:\Users\lnc7> |
```

Figure 18. Negative control: a modern client (GnuTLS) fails the handshake against the same endpoint, confirming the server accepts only the legacy cipher suite required by the PS2.

Database Connectivity Test

The MariaDB connection was tested by executing a direct query from within the container using the application-level credentials.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

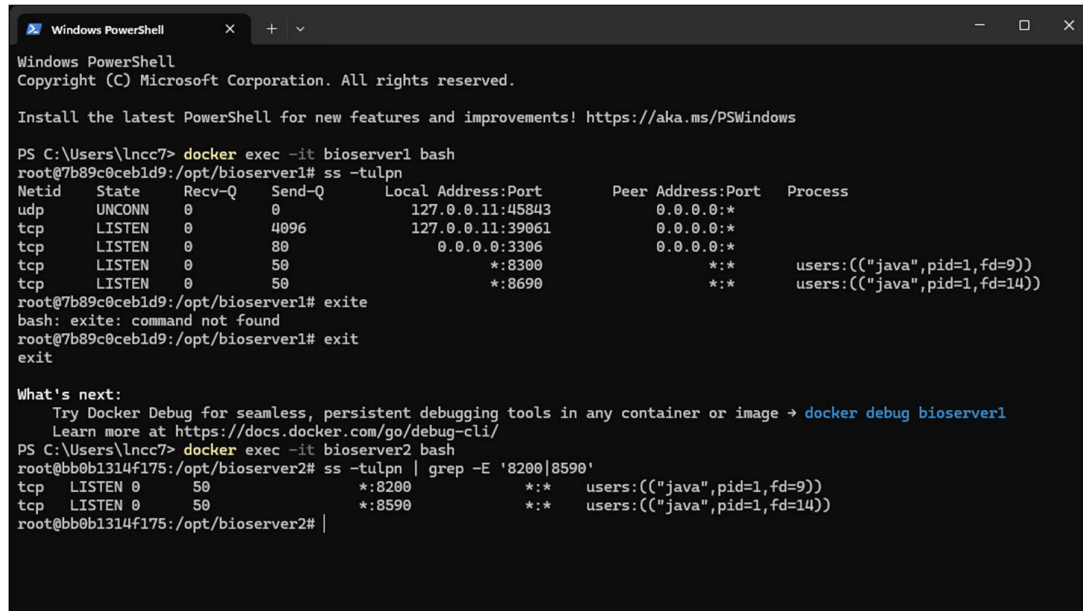
PS C:\Users\lnc7> docker exec -it bioserver1 mariadb -u bioserver -p'123456' -e "USE bioserver; SELECT * FROM users;"
+-----+-----+
| userid | passwd |
+-----+-----+
| ab12   | ab12   |
| rnr    | rnr     |
| romry  | 12ab   |
+-----+-----+

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug bioserver1
  Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\lnc7> |
```

Figure 19. Direct MariaDB query executed inside the bioserver1 container, returning the registered user records.

Java Server Test

The Java lobby and game servers were tested by verifying that they were listening on the correct ports after startup.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\lncc7> docker exec -it bioserver1 bash
root@7b89c0ceb1d9:/opt/bioserver1# ss -tulnp
Netid      State      Recv-Q     Send-Q      Local Address:Port      Peer Address:Port      Process
udp        UNCONN    0           0           127.0.0.11:45843         0.0.0.0:*
tcp        LISTEN    0          4096         127.0.0.11:39061         0.0.0.0:*
tcp        LISTEN    0           80           0.0.0.0:3306             0.0.0.0:*
tcp        LISTEN    0           50            *:8300                  *:8300                  users:((("java",pid=1,fd=9))
tcp        LISTEN    0           50            *:8690                  *:8690                  users:((("java",pid=1,fd=14))

root@7b89c0ceb1d9:/opt/bioserver1# exit
bash: exit: command not found
root@7b89c0ceb1d9:/opt/bioserver1# exit
exit

What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug bioserver1
  Learn more at https://docs.docker.com/go/debug-cli/

PS C:\Users\lncc7> docker exec -it bioserver2 bash
root@bb0b1314f175:/opt/bioserver2# ss -tulnp | grep -E '8200|8590'
tcp        LISTEN    0           50            *:8200                  *:8200                  users:((("java",pid=1,fd=9))
tcp        LISTEN    0           50            *:8590                  *:8590                  users:((("java",pid=1,fd=14))

root@bb0b1314f175:/opt/bioserver2#
```

Figure 20. ss output confirming the Java servers listening on ports 8300/8690 (File #1) and 8200/8590 (File #2).

5.2.2 Integration Testing

Integration testing verified that all components worked together correctly to handle a complete PS2 connection sequence. Testing was performed using PCSX2 on a local machine with the Internal DNS configured to point to the server.

Test 1: DNAS Authentication

Table 5.1. Integration test 1 — DNAS authentication.

Step	Expected Result	Actual Result	Status
Game contacts DNAS	Server returns 200 OK with a 328-byte response	200 OK returned	Pass
Game contacts v2.5_others	Server returns 200 OK	200 OK returned	Pass

The corresponding server log entry is shown in Figure 8.

Test 2: Matchmaking — Account Creation

Table 5.2. Integration test 2 — Matchmaking and account creation.

Step	Expected Result	Actual Result	Status
Game loads CRS-top.jsp	Server returns 200 OK	200 OK returned	Pass
Game loads login.php	Login page displayed	Page displayed correctly	Pass
Player submits new account	Account created in database	Failed on first attempt — see 5.2.4	Pass after fix
Game loads startsession.php	Session ID returned	200 OK with session data	Pass
Game loads enterareas.html	Lobby server address returned	200 OK with server address	Pass

Test 3: Lobby Server Connection

Table 5.3. Integration test 3 — Lobby server connection.

Step	Expected Result	Actual Result	Status
Game connects to port 8200	Java server accepts TCP connection	Connection established	Pass
Java server processes login	Player session initialized	Server log shows connection	Pass after fix — see 5.2.4
Player appears in lobby	Lobby screen displayed	Lobby accessible	Pass

The same integration test sequence was repeated for Biohazard Outbreak File #1, using ports 8300 and 8690, with equivalent results.

5.2.3 User Acceptance Testing

User acceptance testing was conducted with a remote player connecting through Tailscale VPN from a geographically separate network location, using Biohazard Outbreak File #1, whose address identifier (00000002) appears throughout the corresponding server logs. This represented the ultimate validation of the system.

Test Environment

Table 5.4. User acceptance test environment.

Component	Details
Server Host	Windows 11
Remote Player Device	Windows 11

VPN	Tailscale (WireGuard-based)
Game	Biohazard Outbreak File #1 (Japanese region)
Emulator	PCSX2 2.6.3
Measured VPN latency	~33 ms (peer-to-peer ping measured during the acceptance test session)

Results

Table 5.5. User acceptance test results.

Test	Expected Result	Actual Result	Status
Remote player DNAS authentication	Game passes authentication	Authenticated successfully	Pass
Remote player account login	Login successful	Login successful	Pass
Remote player lobby connection	Player appears in lobby	Player connected	Pass
Both players in same lobby	Players see each other	Players visible	Pass
Online multiplayer session	Game session starts	Session initiated	Pass

The resulting online session is shown in Figure 11.

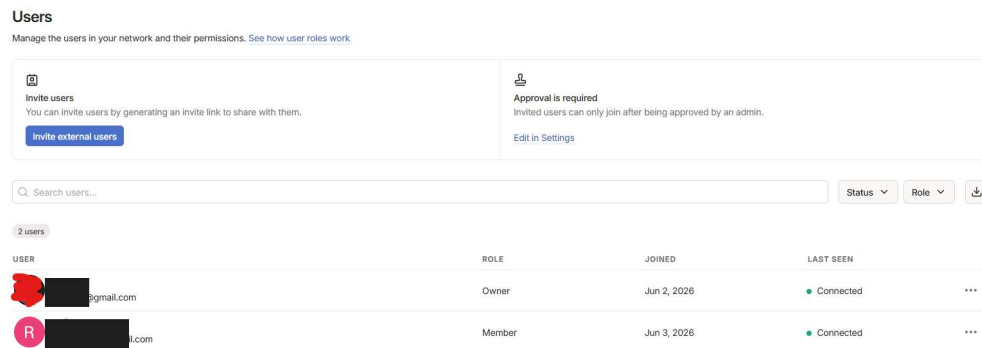


Figure 21. Tailscale admin console showing the two user accounts that make up the private tailnet used for remote play. Email addresses are redacted.

Note on source addresses in the server logs: requests arriving from remote players appear in the Apache access log with a Docker bridge address (for example 172.20.0.1) rather than the player's Tailscale address, because Docker's userland proxy rewrites the source address

when forwarding a published port into the container network. The originating Tailscale addresses are visible instead in the host-side packet capture (Figure 1).

5.2.4 Encountered Issues and Resolutions

This table reflects the actual iterative debugging process, presented in the order the issues were discovered rather than only the final passing state.

Table 5.6. Issues encountered during deployment and testing, with resolutions.

Issue	Cause	Resolution
login_form.php returns HTTP 500	db_cred.php and the pages that include it still used the mysql_* functions, removed in PHP 7.0	Migrated all database calls to the mysqli_* API
Java server throws Access Denied on DB connect	The application user existed only as 'bioserver'@'%', while the JDBC URL connected through localhost — a different host scope, which MariaDB resolves ahead of the wildcard account	Created the account explicitly for both the localhost and % scopes, using MariaDB's IDENTIFIED VIA mysql_native_password syntax
Access Denied persists after the host-scope fix	The upstream project distributed the same placeholder credential across three separate files — bioserver.sql, db_cred.php, and config.properties — each of which had to be edited independently; the Java configuration file was still holding the original placeholder	Aligned all three definitions; this fragmentation is the practical motivation for the secrets-management limitation noted in Chapter 6
Java class fails to compile / load against modern connector	Source code imports com.mysql.jdbc, removed in Connector/J 8.x	Pinned MySQL Connector/J to version 5.1.49
Remote player cannot reach server over VPN	Windows Firewall blocking inbound traffic on the Tailscale interface	Added explicit inbound firewall rules for the game ports
DNAS container cannot query the player database	Each Compose file created its own network, so the container names referenced by the PHP configuration could not be resolved from the DNAS container	Attached the DNAS container to both Bioserver Compose networks; user-defined Compose networks provide internal name resolution, which the default bridge does not

5.3 Conclusion

Testing confirmed that the deployed infrastructure successfully supports the complete PlayStation 2 online connection sequence for Biohazard Outbreak File #1, with equivalent unit- and integration-level results for File #2. All unit tests passed, verifying correct operation of each individual component. Integration testing validated the full connection chain from DNAS authentication through lobby entry, including the specific failures that had to be resolved along the way. User acceptance testing demonstrated that remote players could connect through Tailscale VPN and participate in online multiplayer sessions with a measured latency of approximately 33 ms. The issues encountered during testing were systematic and traceable, and each was resolved through targeted configuration or dependency-version changes without requiring modification to the core server logic.

Chapter 6: Conclusion and Future Work

6. Conclusion and Future Work

6.1 Conclusion

This project successfully deployed a fully functional private online server infrastructure for Biohazard Outbreak File #1 and File #2 on the PlayStation 2 platform, building on an existing open-source codebase (gh0stl1ne, GitLab, AGPL-3.0) rather than developing the protocol emulation from scratch. Nearly two decades after Capcom began retiring the dedicated game servers in 2007, and roughly a decade after Sony's shared DNAS authentication layer was retired in 2016, online multiplayer gameplay for these titles was restored in a private, reproducible deployment and validated through testing with geographically separated players connected over a secure VPN tunnel.

The key technical contributions of this project, relative to the reference codebase and a previously documented VM-based deployment attempt (Chapter 2, Table 2.1), include:

Legacy SSL Compatibility: By compiling Apache HTTP Server 2.4.56 from source and linking it against OpenSSL 1.0.2u, the system establishes SSL connections with the PS2 hardware using the EDH-RSA-DES-CBC3-SHA cipher suite required for DNAS authentication.

PHP Modernization: All database interaction functions were migrated from the deprecated `mysql_*` API to `MySQLi`, restoring full database connectivity without altering the underlying application logic.

Resolved Java Database Driver Conflict: The game server source code requires the `com.mysql.jdbc` package, which was removed in Connector/J 8.x. This was resolved by deliberately pinning the driver to Connector/J 5.1.49 and OpenJDK 11, and by configuring MariaDB to use `mysql_native_password` authentication for compatibility with that driver.

Persistent, Idempotent Deployment: Each container's startup script checks whether the database schema is already initialized before importing it, so restarting or rebuilding a container does not erase player accounts — directly addressing a data-loss issue reported in a prior VM-based deployment of the same codebase.

Partial Multi-Stage Containerization: The Java game server images use a multi-stage Docker build that excludes the JDK and build toolchain from the runtime image. The DNAS image does not yet follow this pattern (see Limitations below).

Secure Remote Connectivity: Tailscale VPN provides an encrypted WireGuard tunnel between the server host and remote players, enabling online multiplayer across different network locations without exposing server ports directly to the public internet.

Network Traffic Analysis: Wireshark packet capture was used throughout the project as the primary diagnostic tool, enabling precise identification of the PS2 connection sequence and the isolation of failures at each layer of the server stack.

6.2 Limitations

The following limitations are acknowledged as deliberate trade-offs or known constraints of the current deployment, rather than oversights:

1. Multiple processes per container: each Bioserver container runs MariaDB, Apache, and the Java runtime together rather than one process per container, because the original server software assumes an always-available local database connection.
2. The DNAS container is not multi-stage: it currently retains its build toolchain (gcc, make, development headers) and uncleaned source archives in the final image, along with an installed but unused MariaDB package, resulting in a larger image than necessary.
3. No unified orchestration: the three server components are defined as independent Docker build/Compose units and must currently be connected to a shared Docker network manually after deployment, rather than through a single top-level Compose file.
4. Region coverage: the DNAS certificates available (cert-jp.pem) cover the Japanese PS2 region only; other regions were not tested.
5. Testing scope: acceptance testing was performed with two simultaneous players, while the original game supports up to four; File #2 remote-play was validated at the integration level but not yet repeated in the full remote acceptance test (see Chapter 5).
6. Secrets management: database credentials are currently set via plain environment variables in the Dockerfile/Compose definitions rather than a dedicated secrets store, which is acceptable for a private, VPN-isolated deployment but would need to change before any public-facing deployment.
7. Legacy cryptography: the 3DES-based cipher suite required by the PS2 hardware is considered weak by modern standards (including known theoretical attacks such as Sweet32) and cannot be avoided without breaking client compatibility; this risk is mitigated by restricting server exposure to a private VPN tunnel rather than the public internet.

8. Password storage: the `users.passwd` field follows the original 2004-era protocol's plaintext-compatible format and was left unmodified to preserve client compatibility, rather than being upgraded to a modern salted-hash scheme.

9. Deployment is bound to one tailnet: the session-server address advertised to clients is written into `config.properties` inside the image as a fixed Tailscale address. Moving the stack to a different host, or a change to the assigned Tailscale address, requires editing that file and rebuilding the image. Reading the value from an environment variable at startup, as is already done for the database credentials, would remove this constraint.

6.3 Future Work and Recommendations

Complete the DNAS container's multi-stage conversion: rebuild it as a two-stage image (build tools in a discarded builder stage; only compiled Apache/OpenSSL binaries and required runtime packages in the final stage), remove the unused MariaDB package, and clean residual source archives from `/tmp` — with before/after image size reported as a concrete metric.

Unify orchestration: consolidate the three independent Compose definitions into a single top-level Compose file with a shared Docker network, so the entire stack starts from one command without a manual network-connect step.

Move secrets out of build files: adopt Docker secrets or an untracked `.env` file for database credentials.

Public Cloud Deployment: deploy the containerized infrastructure on a cloud platform such as AWS or Google Cloud for persistent availability independent of a personal machine being powered on.

Support for Additional PS2 Online Titles: the DNAS authentication infrastructure is not exclusive to Biohazard Outbreak; many other PlayStation 2 online titles used the same Sony DNAS system, making this infrastructure a potential foundation for broader PS2 online game preservation.

Web-Based Server Administration Interface: a web-based administration panel would improve usability for non-technical administrators managing player accounts or server messages.

Enhanced Monitoring: a dedicated log aggregation solution would provide better long-term visibility into server health and player activity than reading Docker logs directly.

References

- [1] M. Piroinen, "Containerization and Cloud Migration of Legacy Web Services," M.Sc. thesis, Aalto University, Espoo, Finland, 2020. [Online]. Available: <https://urn.fi/URN:NBN:fi:aalto-202006213958>
- [2] J. Watada, A. Roy, R. Kadikar, H. Pham, and B. Xu, "Emerging trends, techniques and open issues of containerization: A review," *IEEE Access*, vol. 7, pp. 152443–152472, 2019, doi: 10.1109/ACCESS.2019.2945930.
- [3] P. M. Comparetti, G. Wondracek, C. Kruegel, and E. Kirda, "Prospex: Protocol specification extraction," in *Proc. 30th IEEE Symposium on Security and Privacy*, Oakland, CA, USA, 2009, pp. 110–125.
- [4] gh0stl1ne (dev ghostline), "Bioserver2 — Emulation of the long gone server for Biohazard Outbreak File #2 (PlayStation 2)," GitLab repository, Oct. 2019. Licensed under GNU AGPL-3.0. [Online]. Available: <https://gitlab.com/gh0stl1ne/Bioserver2>. [Accessed: 07-Aug-2026].
- [5] gh0stl1ne (dev ghostline), "Bioserver1 — Emulation of the long gone server for Biohazard Outbreak File #1 (PlayStation 2)," GitLab repository, Oct. 2019. Licensed under GNU AGPL-3.0. [Online]. Available: <https://gitlab.com/gh0stl1ne/Bioserver1>. [Accessed: 07-Aug-2026].
- [6] gh0stl1ne (dev ghostline), "DNASrep — DNAS replacement server," GitLab repository, 2019. Licensed under GNU AGPL-3.0. [Online]. Available: <https://gitlab.com/gh0stl1ne/DNASrep>. [Accessed: 07-Aug-2026].
- [7] obsrv.org community wiki, "Biohazard Outbreak client (PCSX2/PS2) configuration guide." [Online, community-maintained resource].
- [8] Corbin, "Biohazard: Outbreak Server Setup," corbin.zip, Mar. 4, 2023, updated Mar. 25, 2023. [Online]. Available: <https://www.corbin.zip/outbreak-server/>. [Accessed: 08-Aug-2026].

List of Figures

Figure 1. Wireshark packet capture of the PS2 DNAS authentication exchange on TCP port 443, showing the Client Hello and the TLSv1 handshake. (Chapter 3)

Figure 2. Sequence diagram of the PS2 client connection process, from DNAS authentication through lobby entry. (Chapter 3)

Figure 3. Use case diagram for player interaction with the server system. (Chapter 3)

Figure 4. Context / network topology diagram showing the DNAS container, per-title game server containers, Docker networking, and the Tailscale VPN tunnel to remote players. (Chapter 3)

Figure 5. Docker Desktop showing all server containers running: my-dnas-server, bioserver1, and bioserver2. (Chapter 3)

Figure 6. PCSX2 Network settings with the Internal DNS entries mapped to the Tailscale address of the server host. (Chapter 3)

Figure 7. Biohazard Outbreak login result screen in PCSX2 confirming a successful account login. (Chapter 3)

Figure 8. Apache access log showing successful DNAS and matchmaking requests returning HTTP 200. (Chapter 3)

Figure 9. Biohazard Outbreak login page in PCSX2 showing the ID and Password fields for both existing and new accounts. (Chapter 4)

Figure 10. Room creation screen served by the Java lobby server, showing the configurable session parameters: player count, wait limit, difficulty, friendly fire, scenario, and character. (Chapter 4)

Figure 11. Two players connected to the same online session, shown at the character selection stage. (Chapter 4)

Figure 12. Entity-relationship diagram of the player database schema. (Chapter 4)

Figure 13. MariaDB databases and tables inside the bioserver1 container, listed via docker exec. (Chapter 4)

Figure 14. docker-compose.yml for bioserver1, showing the service definition, environment variables, and port mappings. (Chapter 4)

Figure 15. Dockerfile showing the two-stage build: the JAR builder stage and the slim runtime image stage. (Chapter 4)

Figure 16. Startup log of bioserver2 showing MariaDB, Apache, and the Java game server initialising, including the schema-already-initialised skip message. (Chapter 4)

Figure 17. DNAS SSL debug log confirming the negotiated legacy cipher suite EDH-RSA-DES-CBC3-SHA over TLSv1. (Chapter 5)

Figure 18. Negative control: a modern client (GnuTLS) fails the handshake against the same endpoint, confirming the server accepts only the legacy cipher suite required by the PS2. (Chapter 5)

Figure 19. Direct MariaDB query executed inside the bioserver1 container, returning the registered user records. (Chapter 5)

Figure 20. ss output confirming the Java servers listening on ports 8300/8690 (File #1) and 8200/8590 (File #2). (Chapter 5)

Figure 21. Tailscale admin console showing the two user accounts that make up the private tailnet used for remote play. Email addresses are redacted. (Chapter 5)

List of Tables

Table 2.1. Comparison between the prior VM-based reference deployment and this Docker-based deployment.

Table 3.1. Functional requirements of the server infrastructure.

Table 3.2. Non-functional requirements and their associated constraints.

Table 3.3. Use case 1 — Player authentication via DNAS.

Table 3.4. Use case 2 — Player login.

Table 3.5. Use case 3 — Create new account.

Table 3.6. Use case 4 — Join game lobby.

Table 4.1. Tools, technologies, and versions used in the deployment.

Table 4.2. Structure of the sessions table.

Table 4.3. Structure of the users table.

Table 4.4. Structure of the hnpairs table.

Table 4.5. Structure of the motd table.

Table 4.6. Composition of Container 1 (my-dnas-server).

Table 4.7. Composition of Container 2 (bioserver2).

Table 4.8. Composition of Container 3 (bioserver1).

Table 5.1. Integration test 1 — DNAS authentication.

Table 5.2. Integration test 2 — Matchmaking and account creation.

Table 5.3. Integration test 3 — Lobby server connection.

Table 5.4. User acceptance test environment.

Table 5.5. User acceptance test results.

Table 5.6. Issues encountered during deployment and testing, with resolutions.

Appendix

A. PPT

B. Materials

[Docker configuration files (Dockerfile, docker-compose.yml, docker-entrypoint.sh)



Appendix file.zip